thws Technische Hochschule Würzburg-Schweinfurt

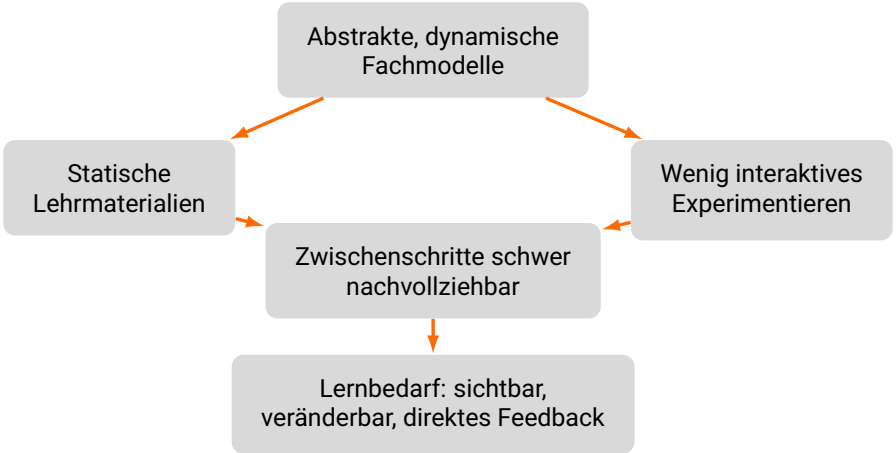
Projektarbeit

INTERAKTIVE LERNPLATTFORM FÜR GRUNDLAGEN INFORMATIK

M. Nöth, R. Schröder, E. Ebert, D. T. Pham, K. Popov

1. KONTEXT UND ZIELE

PROBLEMSTELLUNG



thws
Technische Hochschule
Würzburg-Schweinfurt

ZIELE DER PLATTFORM

- Browserbasierte Lern- und Übungsplattform
- Überblick, Beispiele, Übungen und freie Simulation
- Einheitliche Bedienung, Mehrsprachig und helle / dunkel Darstellung

2. ANFORDERUNGEN UND PROJEKTABLAUF

ANFORDERUNGEN

- Didaktik: interaktive Simulatoren und direktes Feedback
- Bedienung: intuitiv, einheitlich und konsistent
- Technik: modularer Aufbau / Erweiterbarkeit, browserbasiert, Teilbarkeit
- Qualität: Wartbarkeit, fachliche Korrektheit und Reproduzierbarkeit, sinnvoller Umgang mit KI

thws
Technische Hochschule
Würzburg-Schweinfurt

ZUSAMMENARBEIT UND FEEDBACK-ZYKLEN

- Agil orientierte, iterative Arbeitsweise
- Wöchentliches Teammeeting
- Git, GitLab, Work Items und Milestones
- GitLab CI Pipeline mit Raspberry Pi Runner
- Regelmäßiges Auftraggeber-Feedback

3. LÖSUNGSANSATZ: ARCHITEKTUR UND DESIGN

TECHNISCHE BASIS

- React 19 + TypeScript + React Router
- Vite + Tailwind CSS 4 + shadcn/ui
- Zustand + i18next
- Monaco + OpenJDK javac + TeaVM

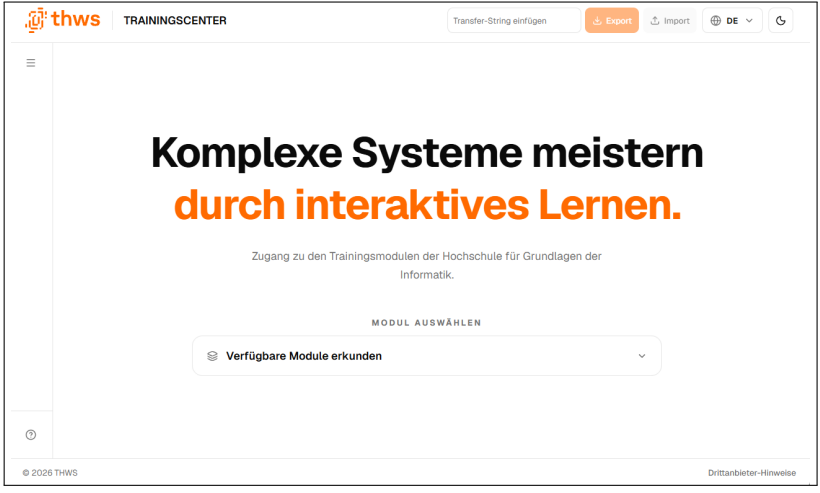
MODULARE PLATTFORMARCHITEKTUR

- Fachlich getrennte Topics
- Gemeinsame Shared Components
- `moduleCatalog.tsx` als zentrale Modulverwaltung

UNIFIED UI: WIEDERKEHRENDES DESIGN

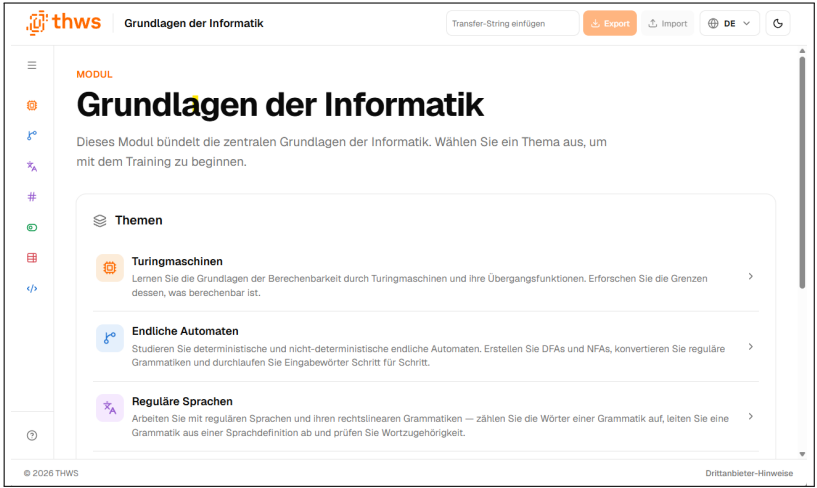
- Header, Sidebar, Footer und Content
- Pro Topic: Überblick, Übungen, Beispiele und Simulation
- Gemeinsame Komponenten
- Fachspezifische Visualisierung in jedem Topic

UI – MAINPAGE

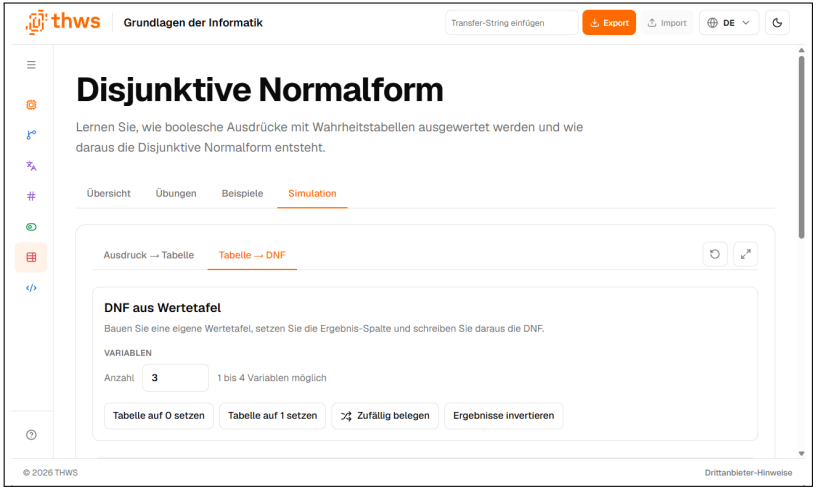

thws
TRAININGSCENTER
Transfer-String einfügen
Export
Import
DE
Komplexe Systeme meistern
durch interaktives Lernen.
Zugang zu den Trainingsmodulen der Hochschule für Grundlagen der Informatik.
MODUL AUSWÄHLEN
Verfügbare Module erkunden
© 2026 THWS
Drittanbieter-Hinweise

thws
Technische Hochschule
Würzburg-Schweinfurt

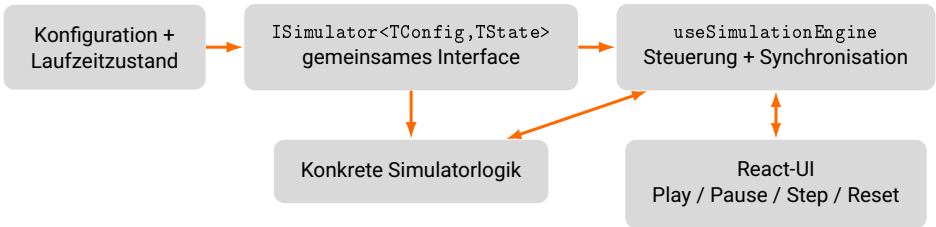
UI – MODULPAGE



UI – TOPICPAGE SIMULATOR



SIMULATOR-ARCHITEKTUR



IMPORT, EXPORT UND LOKALE PERSISTENZ

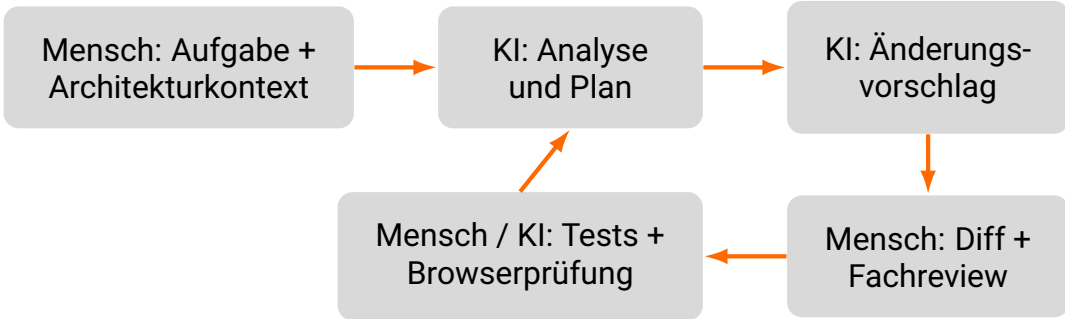
- Einheitliches Transfer-Codec Interface
- Kompakte Base64URL-Transferstrings (12Bit topicStorageId + 6 Bit taskStorageId + Payload)
- Kollisionsfreie lokale Persistenz

4. KI-EINSATZ IM PROJEKT

KI-WERKZEUGE IM PROJEKT

- Stitch, ChatGPT, Gemini, Claude, Codex, Antigravity, verschiedene OpenCode und lokale Modelle
- MCP, Graphify und Agent Skills

ARBEITSWEISE: KI ALS CO-DEVELOPER



GRENZEN UND ERKENNTNISSE BEIM KI-EINSATZ

- Kontextverlust bei großen Aufgaben → `handover.md`
- Duplikate und toter Code → Code Review mit KI
- Nutzen durch kleine, testbare Schritte

5. QUALITÄT UND DEPLOYMENT-STRATEGIE

QUALITÄTSSICHERUNG

- ESLint + Prettier + fallow
- Vitest + Playwright + MCP
- Vite Build
- Lighthouse + manuelles Review

DEPLOYMENT

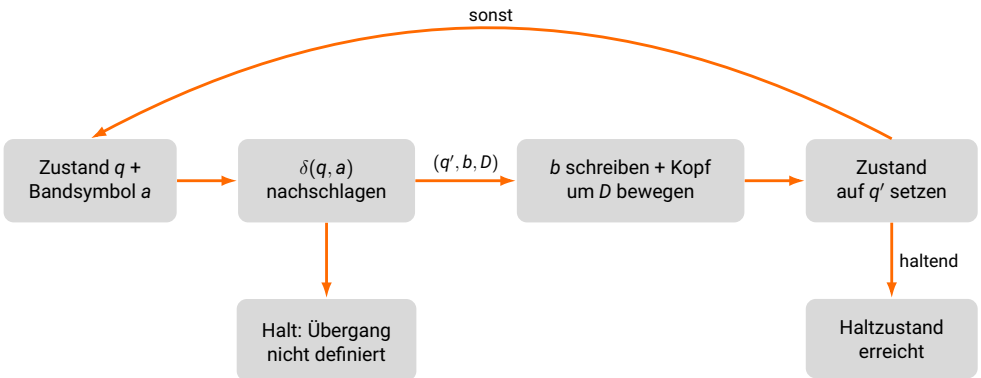
- Statischer Vite-Build als ZIP ausgeliefert
- Deployment Skript für automatischen GitLab release
- Browserbetrieb durch lokalen Server
- Online hosting durch Auftraggeber abgelehnt

FACHMODUL: TURINGMASCHINE

TURINGMASCHINE KONFIGURIEREN

- Übergänge als (q', b, D) in der Übergangstabelle
- Zustände, Haltezustände und Anfangsband definierbar
- Eingabe- und Bandalphabet werden automatisch abgeleitet
- Partielle Übergangsfunktionen: Leere Zellen führen zum Halt

SCHRITTAUSFÜHRUNG

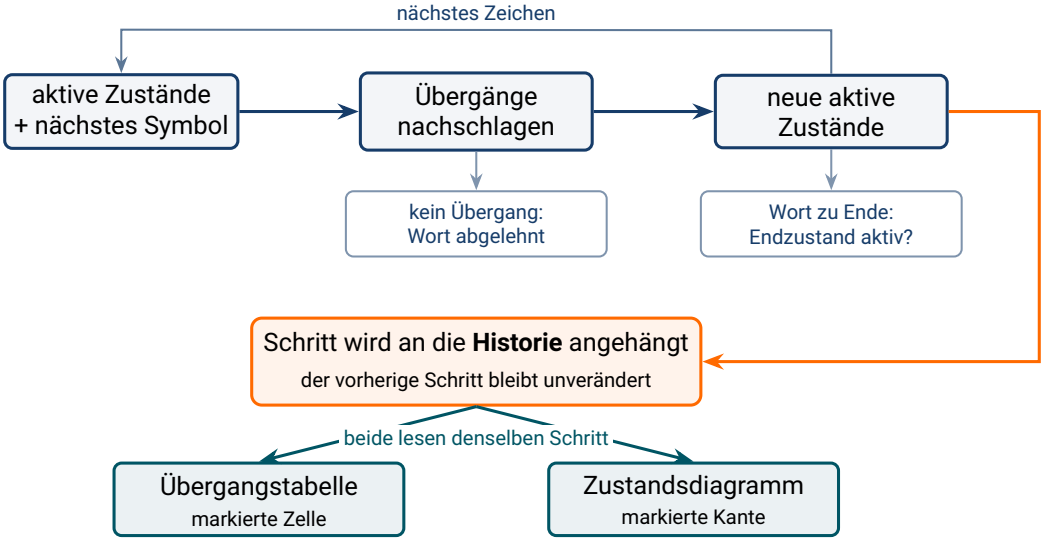


FACHMODUL: ENDLICHE AUTOMATEN

ENDLICHE AUTOMATEN – ÜBERBLICK

- **DFA** und **NFA** selbst definieren
- **Rechtslineare Grammatik** $\rightarrow$ **Automat**
- Zwei Sichten: **Übergangstabelle** und **Zustandsdiagramm**
- **Stapeltest** für mehrere Wörter gleichzeitig
- **Schritt-Historie**: vorwärts, rückwärts, zu einem früheren Schritt
- **Übungen** mit Rückmeldung: korrekt, falsch, Gegenbeispiel

SIMULATIONSSCHRITT

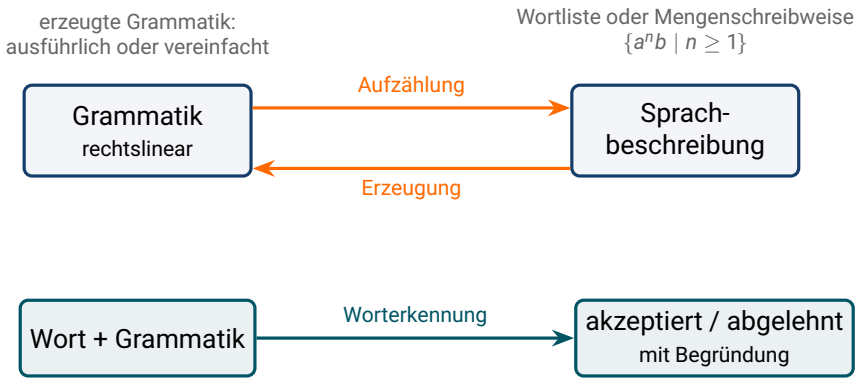


FACHMODUL: REGULÄRE SPRACHEN

REGULÄRE SPRACHEN – ÜBERBLICK

- **Rechtslineare Grammatik** selbst schreiben – geprüft schon beim Tippen
- Drei Modi: **Aufzählung**, **Erzeugung**, **Worterkennung**
- **Übungen** mit Rückmeldung: fehlende und falsche Wörter

REGULÄRE SPRACHEN – DREI ARBEITSWEISEN

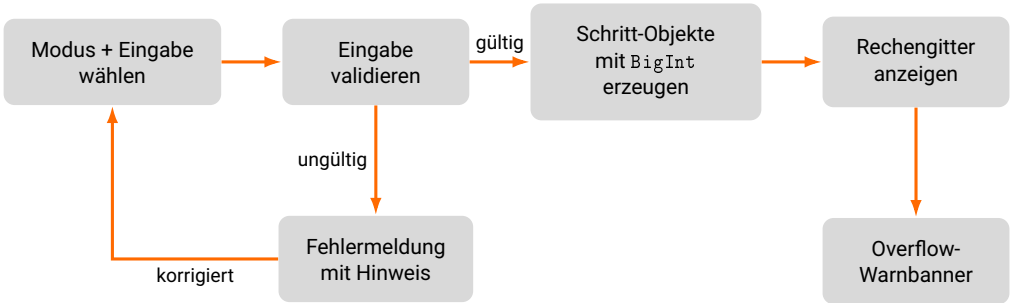


FACHMODUL: ZAHLENSYSTEME

ZAHLENSYSTEME KONFIGURIEREN

- Fünf Modi: *Integers*, *Negative*, *Fractional*, *IEEE-754* und *Arithmetik*
- Eingaben je nach Basis mit Regex-Filtern validiert, Bitlänge explizit begrenzt
- Negative Zahlen in fester 8-Bit-Breite (Wertebereich −128 bis 127, abgestimmt auf die Vorlesung)
- Berechnungen durchgängig mit `BigInt` statt `Number`: keine Rundungsfehler bei großen Zahlen

SCHRITTAUSFÜHRUNG

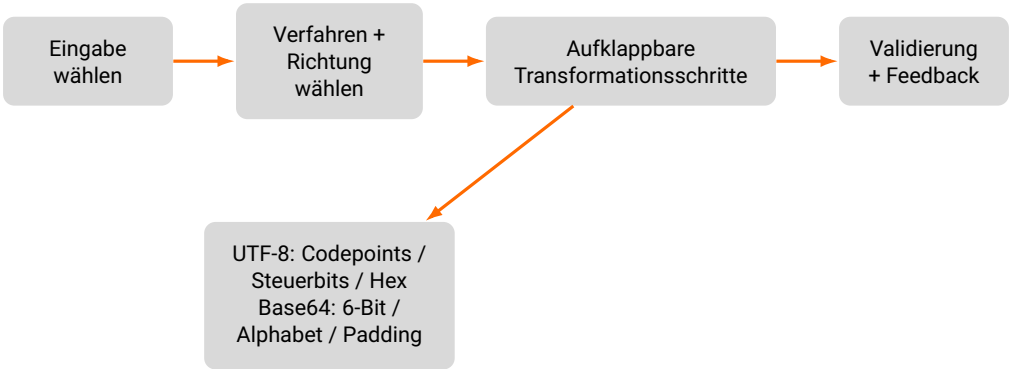


FACHMODUL: KODIERUNG

KODIERUNG KONFIGURIEREN

- Zwei Verfahren: **UTF-8** und **Base64**
- Freier Text oder dynamische Übung als Eingabe
- Kodieren und Dekodieren in beide Richtungen möglich
- Aufklappbare Zwischenschritte mit Farbcodierung
- Dynamische Übungen nach Schwierigkeit (easy / medium / hard)

ABLAUF IN DER APP



Jeder Zwischenschritt ist aufklappbar – Studierende entscheiden selbst, wie tief sie einsteigen.

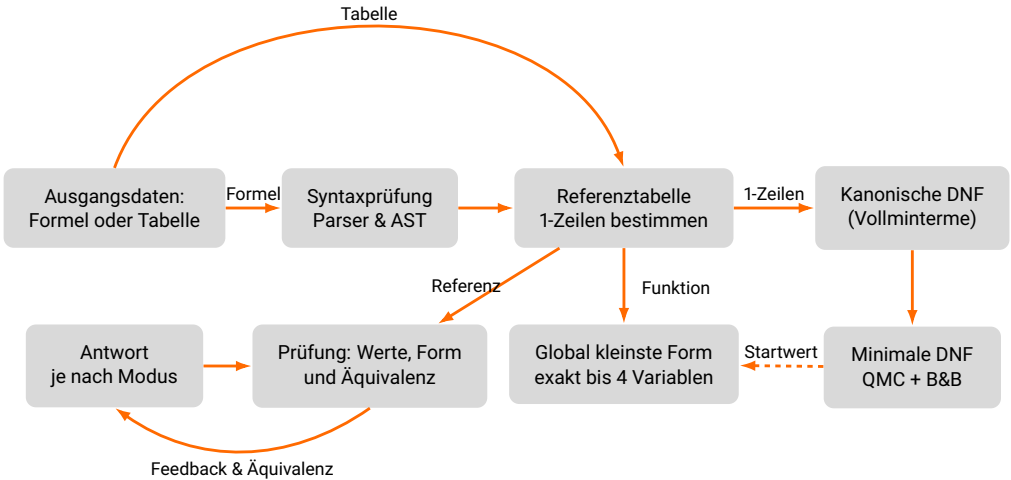
Bewusst keine Animationen: Schritte bleiben statisch und im eigenen Tempo nachvollziehbar.

FACHMODUL: DISJUNKTIVE NORMALFORM

DISJUNKTIVE NORMALFORM – ÜBERBLICK

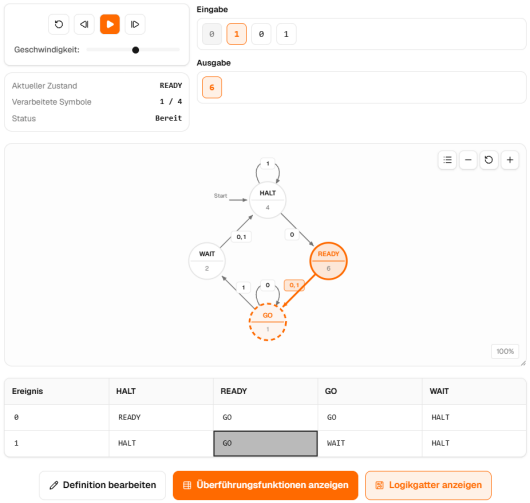
- Zwei Aufgabenmodi: Ausdruck → Wahrheitstabelle und Wahrheitstabelle → DNF
- Drei Ergebnisstufen: kanonische DNF, minimale DNF und optional global kleinste UND/ODER-Formel
- Minimierung mit Quine-McCluskey und Branch-and-Bound
- Semantische Prüfung erkennt logische Äquivalenz
- Begrenzung auf maximal 4 Variablen für exakte Synthese

SCHRITTAUSFÜHRUNG



FACHMODUL: MOORE MEALY

INTERAKTIVE SIMULATOR DARSTELLUNG



- Automaten frei konfigurieren
- Eingaben Schritt für Schritt simulierbar
- Aktueller Zustand und Übergang direkt sichtbar
- Vor- und Zurückspringen in der Simulation

AUTOMAT ANALYSIEREN & SCHALTUNG DARSTELLEN

Automat & Übergänge

- Zustände und Übergänge definieren
- Ausgabe abhängig von Moore bzw. Mealy

Wahrheitstabelle

- Zustände und Eingaben binär codieren
- Boolesche Funktionen automatisch ableiten
- DNF minimieren

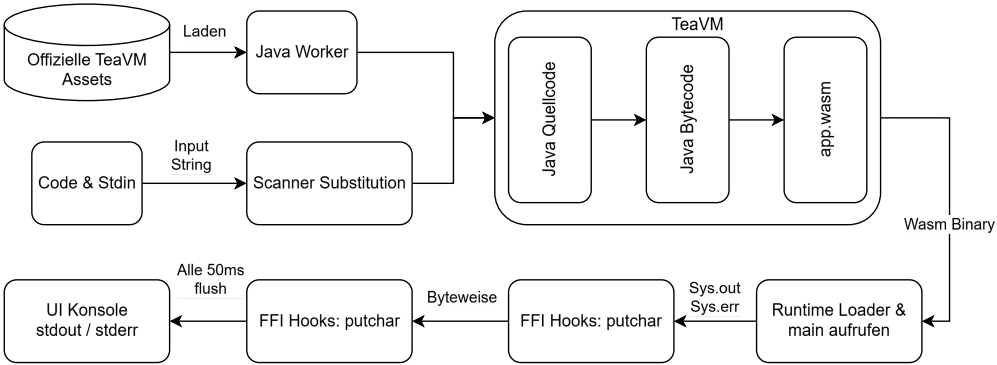
Logikgatter

- Funktionen als Schaltung visualisieren
- Gatterdarstellung
- Verbindung von Automat und Hardware

→ **logische bzw. hardwareseitige Umsetzung sichtbar machen**

FACHMODUL: JAVA

JAVA – ÜBERBLICK



BESONDERHEITEN UND EINSCHRÄNKUNGEN

- **Offline nutzbar** – Verteilung als ZIP, keine externe Verbindung nötig
- **Kein lokales Java** – kein JDK, Gradle oder Git erforderlich
- **Kein Java-Backend** – Kompilierung und Ausführung lokal im Browser
- **Plattformunabhängig** – Nutzung auch auf Tablets / iPads im Browser
- **Browser-Sandbox** – kein interaktives `System.in`, kein Dateisystemzugriff
- **Für Lernaufgaben ausreichend** – Code, (Eingaben) und `stdout`/`stderr`

6. INTERAKTIVE LIVE-DEMO

DEMO-ZIEL: DEN LERNWEG SICHTBAR MACHEN

- Eingabe verändern
- Schrittweise ausführen
- Zwischenschritte erklären

7. AUSBLICK

PERSPEKTIVEN UND OFFENE PUNKTE

- Weitere Lernbereiche und Topics
- Aufgabenverwaltung und Aufgabenvarianten
- Mobile Nutzung
- Java Zeilensperrung implementieren
- Lernstandsverfolgung
- Gamification
- LMS-Anbindung (E-Learning)